

Don't Let Your Agent Read Everything : Erasing an Agent's Over-fetched Data

Abstract—Agentic AI is now using our software for us, and at the core is Anthropic’s Model Context Protocol (MCP). As MCP provides a great deal of convenience for agentic workflows, Software-as-a-Service (SaaS) companies have started to provide “MCP Compatibility (e.g., API)” to their products, as a means to adapt and survive the agentic era. However, as SaaS companies are burdened with providing easy, widely compatible ways to connect with agents, this has led to over-privileged agents reading and writing unintended sensitive data. In order to prevent the agent from accessing such sensitive data, we propose Contextual Leak Evaluation and Redaction (a.k.a CLEAR), a security layer that sits in between the agent and the MCP server and redacts sensitive data, providing a unique way to insert agent safety and governance. By integrating CLEAR into the agent workflow, we have achieved 80% successful redaction across 100 custom data over-retrieval scenarios, with an average of 14.6 seconds added to the overall workflow. CLEAR also demonstrated a $7.4\times$ higher median k -anonymity score compared to unprotected retrieval and $3.6\times$ higher than traditional PII-boundary masking (using Microsoft Presidio DLP), evaluated against a simulated 3-step cumulative identity reconstruction attack on a 1,000,000-record synthetic persona dataset—reducing the rate of extreme re-identification scenarios ($k \leq 10$) from 4.7% to 0.9%, a $5.2\times$ reduction. This research lays the ground for dynamically controlling which data reaches the AI model, thus limiting what the AI can say or do with the controlled data. This provides an insight into systematic harnessing, a true structural way of controlling agent behavior.

Index Terms—Agentic AI, AI Security, Agent Security, CLEAR

I. INTRODUCTION

Today, we are living in the “Agentic Era”. Large Language Models (LLMs) have now turned into agents, where they act, work, and perform on behalf of humans. And with this technology, we are rapidly moving away from traditional human-centric software toward an “Agentic-driven” ecosystem.

At the center of this transition is the widespread adoption of standardized integration frameworks, most notably Anthropic’s Model Context Protocol (MCP) [1]. MCP provides a unified abstraction layer that allows LLMs to seamlessly connect with external data sources and third-party tools, and with the significant trend of using AI in almost any type of field, the abstraction and convenience of MCP appealed to a majority of application developers. The drastic reduction in integration friction and the accessibility of so many different tools created a strong network effect and forced the software industry to open its doors to autonomous AI users.

To survive and adapt in this new agentic era, Software-as-a-Service (SaaS) companies are aggressively refactoring their products to offer “MCP Compatibility” or dedicated MCP server APIs. For enterprise SaaS providers, making their

platforms easily readable and writeable by autonomous agents has quickly shifted from a premium feature to a core baseline for market survival.

However, this push for agent connectivity has inadvertently bypassed a fundamental principle of security: the principle of least privilege. Because SaaS providers are burdened with making their integration endpoints as easy to use as possible, they often expose broad data surfaces to agents. This approach has led to “over-privileged agents”, authorizing them to perform general tasks—such as drafting a meeting summary—frequently gaining the structural ability to read sensitive data (e.g., financial spreadsheets, personal credentials, or proprietary meeting materials) that was completely unintended for the AI model to see.

The core of this issue lies in the lack of contextual awareness in modern security guardrails. Most existing Data Loss Prevention (DLP) technologies are designed to inspect data exclusively at the final input and output boundaries—the direct interaction points between the user and the LLM. And more often than not, developers who attempt to secure tool-using agents often naively transplant these exact boundary-filtering techniques directly onto the tool call layer. This approach fails fundamentally because traditional tools rely on static keyword matching or rigid regular expressions, which completely strip away the semantic intent behind a live tool invocation. As a result, these legacy tools either disrupt legitimate workflows by triggering excessive false positives on benign technical data, or fail entirely to recognize indirect data leaks caused by malicious prompt injections. Ultimately, to safely deploy and utilize autonomous agents in production, we must establish a way to rigorously govern their autonomous behaviors. Achieving control over an agent’s actions requires pinpointing the exact junction in the workflow where the context of its behavior can be fully captured.

Our proposed framework, CLEAR, directly embodies this new method of control. Rather than treating the agent as a black box and trying to fix its reasoning or prompts, CLEAR shifts the defense focus to the agent’s workflow layer, and by doing so achieve direct workflow governance. We introduce a judgment layer and a filtering layer upon a gateway that resides between the agent and the MCP servers, built specifically for intercepting and analyzing the user’s requested actions, the semantic context of consequent tool calls, and the final result of the tool calls. By living inside the tool-execution loop, CLEAR continuously tracks which data was needed, how it was retrieved, and where it came from, then masks or trims unnecessary data according to the evaluated semantic context.

This architectural positioning allows CLEAR to dynamically remove sensitive data before it ever reaches the underlying model, providing protection against data leakage caused by autonomous agents or malicious actors.

In summary, the primary contributions of this paper are as follows:

- **Conceptual Framework:** We define a runtime security model for LLM agents that shifts the defense focus from prompt-layer security to workflow-layer governance. This framework addresses the critical gap where probabilistic models fail to enforce minimum data collection.
- **System Architecture:** We design and implement CLEAR, a workflow-native data protection architecture for tool-augmented agents. By inserting a gateway interceptor within the tool-execution loop, the system analyzes the semantic context of active tool calls, dynamically redacts and deletes unnecessary data, and tracks data provenance in real time.
- **Empirical Evaluation:** We conduct a comprehensive evaluation of CLEAR across 100 diverse agent execution scenarios. Our results demonstrate that CLEAR effectively mitigates data over-exposure, providing robust data redaction with acceptable downtime, performance, and token overhead in enterprise environments.

II. RELATED WORK

A. Traditional DLP Solutions in Agentic LLM Architectures

To prevent sensitive data leaks in language model deployment, enterprises have traditionally relied on standard DLP solutions. These legacy systems are primarily designed to monitor data at the final input and output boundaries—specifically inspecting the direct text prompt sent by the user or the final text response generated by the LLM. They operate by using static keyword matching and regular expressions to detect and block signatures of sensitive data, such as personally identifiable information (PII) or corporate secrets. This is directly derived from legacy security tools that have exact schemas for what they monitor (e.g., network packets, file hashes, etc).

However, as LLMs became autonomous agents, and autonomous agents began to utilize external software tools via protocols like MCP, traditional DLP solutions were unable to prevent data loss that occurs in between the input and the output level. Moreover, in order to address this problem, developers have naively transplanted these exact boundary-filtering techniques directly onto the tool-calling layer. This approach introduces a structural limitation. Since traditional DLP tools are fundamentally context-unaware, they evaluate data payloads in isolation, stripping away the semantic intent behind a live tool call. This raises many false alarms, which ultimately leads to unavailability; the very point that many people consider crucial in using agents.

B. Emerging Privacy Threats in Agentic LLM Architectures

Recent literature claims that traditional output-centric safety measures inherently fail in agentic environments, where an

agent can generate a benign response while executing dangerous, unauthorized actions through its tool interfaces without the consent and knowledge of the user. These behavioral risks manifest through two main factors: external malicious manipulation and internal autonomous over-inference.

First, external malicious manipulation refers to any type of interference, from an unauthorized third party, that shifts the original workflow of the agent to benefit the said third party. Lan et al. formalized the threat of “Silent Egress”, demonstrating that a malicious instruction hidden in URLs or metadata can trick an agent into giving up sensitive data to the attacker [2]. They have proved that prompt-layer hardening or delimiter tags offer highly limited protection, as LLMs naturally struggle to separate control command from low-privilege web data within a flattened context window.

Second, internal autonomous over-inference refers to the tendency where a model, driven by sycophancy, proactively over-collects data and aggressively synthesizes them to infer sensitive, private data that the user did not intend to disclose. Qiao et al. investigated “Tools Orchestration Privacy Risk (TOP-R)”, a dynamic evolution of the classic “Mosaic Effect” in the agentic era [3]. TOP-R is caused by an agent that proactively collects scattered, non-sensitive data fragments across multiple tools, and aggressively synthesizes the scattered pieces of data to infer sensitive information that the user never intended to disclose. Complementing this conceptual framework, Zhang et al. empirically demonstrated through their PrivacyPeek benchmark that LLM-based agents routinely over-acquire sensitive information beyond task scope across diverse application domains, and crucially, that prompt-level defenses reduce only a fraction of this acquisition-stage leakage, underscoring the need for architectural mitigation [4].

Although prompt-level mitigation strategies such as the Privacy Enhancement Principle (PEP) attempt to suppress internal knowledge synthesis, and automated frameworks such as AgentRaft provide vital diagnostic capabilities, critical architectural defense gaps remain in live environments [5]. More recent work has begun exploring architectural interventions at the agent–tool boundary: Ma and Schmid proposed SecureClaw, a dual-boundary gateway that replaces raw sensitive values with opaque handles and enforces a PREVIEW→COMMIT protocol for write operations [6]. While demonstrating the viability of gateway-based control, SecureClaw’s handle-based confinement relies on bounded summaries as a declassification interface, a fundamentally different approach from semantic-context-aware redaction.

Post-hoc auditing cannot prevent the unintended runtime actions of an agent, and soft prompt constraints cannot fundamentally fix the behavioral defects of the underlying model itself. It is crucial to fill these gaps by deploying a fundamentally different monitoring paradigm; one that is natively designed around agentic workflows that possesses the structural capability to oversee and control the agent’s actions based on semantic context and intent.

III. BACKGROUND & PROBLEM STATEMENT

In legacy web architectures, data isolation and privilege separation were governed by deterministic access control lists (ACLs) enforced at the server side. However, autonomous agents operate as probabilistic engines that dynamically chain tools and pass data packets based on runtime reasoning. This means that the agent itself is a non-deterministic entity that static security boundaries do not work. We cannot simply look at a runtime tool call in isolation and determine if it is safe. We must evaluate the agent’s behavior against the user’s request, and check the alignment with what the user originally permitted.

Therefore, it is crucial to establish a formal model that explicitly links user intentions to runtime tool interactions. Without this connection, a security system cannot separate a legitimate, necessary action from unintended data over-collection or malicious behavior. This section formalizes the system model of tool-augmented agents, defines the core primitives of our contextual evaluation, and outlines the specific threat landscape that CLEAR aims to mitigate.

A. System Model

Rather than treating the agent as an isolated black box, we formalize the execution space of an agentic workflow through three sequential operational boundaries that represent the life cycle of a single execution turn: the Input boundary (I), the Tool interface (T), and the Result payload (R).

- **The Input Boundary (I):** This represents the semantic scope and data constraints explicitly defined by the user’s intent within the current session turn. I contains the minimum set of target information fields and destination parameters required to fulfill a benign user goal G (e.g., extracting a specific identifier and routing it to a designated endpoint). *For the sake of narrative simplicity, this boundary is used interchangeably with the user’s actual “input prompt” throughout the remainder of this paper.*
- **The Tool Interface (T):** This represents the functional capabilities and functional descriptions exposed by available MCP servers. Each tool $t \in T$ acts as an authorized capability gateway (e.g., `read_file`, `send_email`) that accepts structured arguments and maintains technical privileges to interact with underlying databases or external SaaS environments. *In practical deployment terms, this interface directly maps to the specific “API endpoints” or tool-calling structures invoked by the agent.*
- **The Result Payload (R):** This represents the raw, compiled data stream returned by the environment after a tool invocation, or the transactional packet being dispatched toward an external sink. R contains fine-grained data schemas which may include sensitive data that is entirely unmentioned within the initial input boundary. *In subsequent sections, this payload is conceptually referred to as the “runtime data stream” or tool execution output.*

B. Formulations of Data Leak

Based on the definitions of the Input boundary (I), Tool interface (T), and Result payload (R) established in the system model, CLEAR formalizes the structural causes of data over-exposure and malicious exfiltration. Rather than viewing data leaks as random, probabilistic anomalies, we mathematically categorize these vulnerabilities into three distinct operational states based on how these three boundaries intersect during runtime:

- **Category 1: Benign Workflow ($I = T$)** This state formalizes a completely safe and standard operational scenario. Here, the semantic scope of the user’s explicit intent (I) perfectly aligns with the functional capability of the invoked tool (T). Because the agent executes a tool call that exactly matches what the user requested, the runtime execution remains entirely within authorized boundaries, and the Result payload (R) contains only the data expected to fulfill the task. This category serves as the baseline for secure and compliant agent behavior.
- **Category 2: Excessive Data Collection ($I < T$)** In this state, the user’s intent (I) is narrowly defined, but the agent creates tool calls (T) that possess broad read capabilities. Because the tool call pulls the complete, unfiltered data by design, it introduces a significant risk in sensitive data exposure within the Result payload (R). This state is the most common state among the three, as external tool vendors design the tools to be “maximally helpful” so that the agentic workflow is most efficient and convenient.
- **Category 3: Adversarial Logic Hijacking ($I \neq T$)** This state formalizes indirect prompt injection. Here, the user’s actual semantic intent (I) has no logical or operational connection to the tool (T) being invoked ($I \neq T$), thus meaning a malicious attempt was made to hijack or abuse the agent’s workflow. The agent’s reasoning loop is completely hijacked by hidden instructions embedded within the environment data and this tricks the agent into executing unauthorized tools such as routing internal credentials or authentication tokens directly to an outside adversary.

C. Threat Model

We define the threat landscape by outlining attacker types, potential leak surfaces, and our core security assumptions.

- **Attacker Types:** In this work, we consider two distinct types of attackers. The first is an external adversary, who implants malicious instructions in the external tools or environments, with the intent of hijacking the agent’s workflow into exfiltrating PII or sensitive enterprise data. The second is an internal risk source that mainly consists of the agent’s own reasoning engine. In this case, there is no external attacker; rather, the model itself acts inadvertently and autonomously to leak sensitive information, mainly due to misaligned helpfulness and sycophancy.
- **Leak Surfaces:** The primary source of data leak occurs inwardly at the tool-response boundary, where an invoked

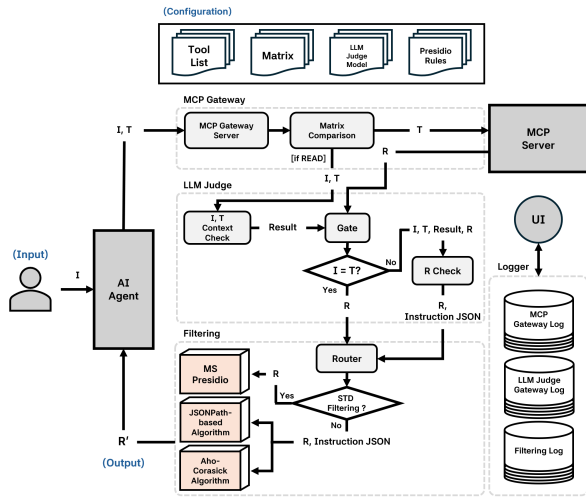


Fig. 1. CLEAR System Architecture

tool returns the complete, raw data that contains unintended sensitive data directly to the underlying AI model. When an over-privileged or maliciously exploited tool pulls a complete, raw data from the external environment, this data is automatically ingested into the LLM’s context window. Once this data is digested by the probabilistic AI model, it is subject to be used in unpredictable agent behavior, therefore leading to a possible data leak.

- **Trust Boundary and Defense Goals:** CLEAR treats both the external SaaS environments and the core LLM reasoning engine as untrusted entities. We assume that the user’s initial input prompt represents the only ground truth of secure intent. The primary goal of CLEAR is to operate as a zero-trust runtime gatekeeper within the workflow layer. We intentionally exclude the mitigation of malicious user-driven actions (such as direct prompt injections or malicious structural writes) from our scope, as countering adversarial user inputs falls strictly within the domain of input-stage “LLM Guardrails”. Our trust boundary isolates and monitors read operations where raw information enters the agentic loop. In order to do so, it must successfully intercept all active tool execution, inspect data payloads, and redact sensitive data before they can be exposed to the underlying AI model or written to an untrusted external source.

IV. CLEAR ARCHITECTURE

To operationalize the deterministic control model described in our system formulations, CLEAR is designed as a modular runtime security framework. Rather than functioning as a passive scanner, we insert the CLEAR framework directly in between the agent and the external tools. The architecture consists of three primary components: the MCP Gateway, the LLM Judge, and the Filtering Layer.

The overall workflow proceeds through a structured lifecycle during a single agent execution turn. First, the MCP Gateway

TABLE I
R INSTRUCTION JSON SCHEMA

Field Name	Type	Description
status	Enum (PENDING, COMPLETED, FAILED)	Status of instruction generation
reason	String or null	Overall decision rationale or failure reason
instructions	Array<Instruction>	List of filtering rules
error_code	Enum or null (Optional)	Error code when a failure occurs

TABLE II
INSTRUCTION OBJECT SCHEMA

Field Name	Type	Description
action	Enum (MASK, TRIM)	Data sanitization method
field	String (JSONPath) or null	Target field path to apply the rule
keyword	String or null	Target string pattern to match
reason	String	Rationale for selecting the rule

captures the user prompt and live tool calls that correlate to the prompt. Second, the captured tool call is routed to the Matrix, where all tool calls are pre-classified into one of the three categories: Broad Read, Atomic Read, and Other (e.g., write, execute, delete, etc). If a tool call is classified as *Other*, CLEAR immediately bypasses the heavy inspection loop and routes the payload through a fast-path return to minimize runtime latency, as the safety of write-oriented transactions is governed by external input-layer guardrails.

Conversely, when the incoming tool call exhibits “Read” attributes (Broad Read or Atomic Read), it is packaged alongside the user prompt and sent directly to the LLM Judge layer. Within this layer, the tool call is first evaluated by an LLM model called the “I/T Context Checker” to determine its contextual alignment. Even though the transaction was initially flagged as a standard data read, if the checker discovers that the tool being executed bears no semantic relevance to the user’s explicit request ($I \neq T$), it flags the behavior as a Category 3 violation (Adversarial Logic Hijacking). In this event, CLEAR instantly drops the pipeline and issues a BLOCK verdict to prevent corrupted external context from infiltrating the agent.

Only when the checker determines the transaction to be a valid but overly permissive data retrieval path—classified as “Category 2 ($I < T$)” will the returned result (R) from the external environment be analyzed for excessive data collection by a separate LLM model called the “R Instruction Creator”. Upon analysis, the R Instruction Creator identifies the overly collected information and creates an instruction in JSON format for the next layer; the Filtering Layer.

The Filtering Layer consists of three components that filter the excessive data from R ; the basic open-source engine of Microsoft Presidio, the JSONPath-based

algorithmic engine to quickly trim unnecessary fields, and the Aho-Corasick-based algorithmic engine to quickly mask the target keywords. The basic engine is only deployed when

the verdict is Category 1 ($I = T$) in case of included PII in R , and the two algorithmic engines are deployed based on the R Instruction JSON created from the LLM Judge layer.

After the filtering engine has redacted or deleted unnecessary data from R into R' , the Filtering Layer returns R' to the agent, ultimately preventing the agent itself from reading, accessing, and using unintended, sensitive data.

A. LLM Judge Layer

This section is dedicated to explaining the operation and workflow of the LLM Judge layer, as it serves as the contextual reasoning engine of CLEAR, responsible for evaluating the semantic alignment between the user's intent and the agent's real-time actions. While the upfront Matrix handles deterministic syntactic filtering based on API types, the LLM Judge layer standardizes dynamic verification through a specialized two-stage pipeline consisting of two independent language model instances: the I/T Context Checker and the R Instruction Creator.

1) *I/T Context Checker*: When a tool call with a *Read* attribute passes through the Matrix, the I/T Context Checker immediately evaluates the transaction before the tool is allowed to execute or return unredacted data to the agent. It takes the user's initial input prompt (I) and the targeted tool interface metadata (T) as its context inputs. The checker evaluates these inputs against the security formulations defined in Section 3.2 to issue an operational verdict:

- **Immediate PASS (Category 1)**: The transaction is recognized as a compliant, benign read path. CLEAR bypasses the heavy trimming process and forwards the payload directly to the Filtering Layer for standard, lightweight PII scanning.
- **Immediate BLOCK (Category 3)**: The transaction is flagged as an adversarial logic hijacking attempt. CLEAR instantly drops the execution pipeline and issues a **BLOCK** verdict, completely preventing the corrupted external data from infiltrating the agent's context window.
- **Trigger R Instruction Pipeline (Category 2)**: The tool call is deemed necessary but structurally over-privileged ($I < T$). This verdict immediately triggers the second stage of the judge layer to handle the excessive data footprint within the incoming result payload (R).

2) *R Instruction Creator*: Upon receiving a Category 2 verdict, the raw observation result (R) fetched from the external MCP environment is routed to the R Instruction Creator instead of being exposed directly to the underlying AI model. The objective of this component is to bridge the gap between probabilistic reasoning and deterministic redaction. Rather than modifying the text stream directly, the model analyzes R to identify the precise boundaries of redundant data that exceed the initial intent boundary (I).

The R Instruction Creator then translates these semantic insights into an actionable, structured filtering instruction blueprint compliant with the schemas defined in Table I and Table II. By outputting a structured JSON array containing targeted JSONPath expressions and exact string patterns for

masking, the creator provides a machine-readable recipe that empowers the algorithmic filtering engines to trim and mask excessive data with high efficiency and consistency.

B. Filtering Layer

The Filtering Layer is responsible for the final deterministic redaction of the CLEAR framework. Instead of passing the raw text result (R) into a language model for editing, we deliberately separate and delegate the reasoning and identifying what to redact to the LLM Judge, and commission the actual execution of the redaction to an algorithm-based filtering engine in order to achieve both strict security guarantees and enterprise-grade efficiency.

The Filtering Layer uses structured inputs created from the LLM Judge layer and utilizes a multi-tiered algorithmic pipeline composed of three specialized engines: the *Basic PII Scanner*, the *JSONPath-Based Engine*, and the *Aho-Corasick-Based Engine*.

- **Basic PII Scanner**: The first component utilizes Microsoft Presidio, an open-source entity detection framework, to handle standard keyword-based or regular expression-based redaction that may be contained in the completely justified, benign R .
- **JSONPath-Based Engine**: When dealing with structural over-exposure triggered under a Category 2 state ($I < T$), parsing text through regex filters is computationally inefficient and brittle. CLEAR addresses this limitation by deploying an algorithmic trimming engine driven by dynamic JSONPath expressions parsed directly from the generated R Instruction metadata (as specified in Table II). When the instruction specifies a **TRIM** action on a designated target field, the engine evaluates the query path against the JSON-formatted result payload (R) and instantly deletes the entire out-of-scope data block. This programmatic truncation ensures that massive, redundant data schemas are pruned at high speeds without corrupting the overall document structure.
- **Aho-Corasick-Based Engine**: To eliminate non-structural sensitive strings or targeted text patterns, CLEAR integrates a high-performance string matching engine based on the Aho-Corasick automaton. When the R Instruction dictates a **MASK** action on specific text patterns or keyword values, these targets are compiled into a finite state machine. The engine evaluates the payload stream in a single, linear scan—maintaining $O(n)$ time complexity relative to the payload length, regardless of the number of target keywords. Matched strings are replaced with deterministic masking tokens (e.g., [REDACTED]), successfully neutralizing privacy risks while maintaining the operational utility of the underlying agentic loop.

V. EVALUATION

In this section, we conduct a quantitative evaluation of the CLEAR framework to assess its defensive efficacy and computational sustainability in enterprise agentic environments. Our evaluation primarily focuses on answering three critical research

TABLE III
CLEAR BENCHMARK SCENARIO SCHEMA

Field Path	Type	Description
test_id	String	Unique scenario identifier
domain	String	Target application domain
description	String	Scenario description
input.userInput	String	User request input
input.expectedIntent	String	Expected inferred intent
mocking_registry. target_mcp_server	String	Target MCP server name
mocking_registry. target_tool	String	Invoked MCP tool
mocking_registry. arguments	String/Object	Tool invocation arguments
mocking_registry. mockResponse	Object	Simulated tool response
evaluation_ground_ truth.itConsistency. result	String	Expected consistency result
evaluation_ground_ truth.itConsistency. reason	String	Consistency evaluation rationale
evaluation_ground_ truth.censoredTargets	Array<String>	Expected censored targets

questions: (1) Can CLEAR consistently prevent structural data over-exposure ($I < T$) and adversarial logic hijacking ($I \neq T$) across diverse SaaS scenarios? (2) Does our workflow-layer intervention effectively mitigate complex identity reconstruction attacks via multi-step information accumulation? and (3) What are the added costs associated with deploying the dual-stage LLM Judge layer?

A. Setup

To provide a comprehensive validation of CLEAR’s multi-layered architecture, the experiment is divided into two independent, distinct experimental track targeted at different threat vectors.

1) *Experiment 1: Semantic Context Alignment Benchmark:* The first experiment evaluates the real-time reasoning and classification capability of the LLM Judge layer when dealing with immediate runtime vulnerabilities. For this benchmark, we created a self-curated dataset consisting of 100 comprehensive scenarios distributed across 5 diverse enterprise application domains: (1) Notion, (2) Local File System, (3) Slack, (4) Discord, and (5) Outlook. Each scenario explicitly encapsulates structured user input requests, correlating live tool calls, mock environment responses, and annotated ground truths for safety verdicts.

Using given inputs and tool execution arguments, CLEAR is tasked with dynamically outputting precise structural classification verdicts—*Fully Benign Workflow* (Category 1), *Excessive Data Collection* (Category 2), or *Adversarial Logic Hijacking* (Category 3)—and compiling the corresponding machine-readable filtering rules in a structured JSON schema without exposing raw payloads to the agent’s context window.

2) *Experiment 2: Demographic k -Anonymity and Mosaic Effect Mitigation:* The second experiment is specifically engineered to mathematically quantify CLEAR’s capability

to suppress the endogenous “Mosaic Effect,” where an autonomous agent gradually uncovers sensitive, undisclosed private properties by synthesizing non-sensitive information fragments scattered across multiple separate tool responses.

The experimental infrastructure for this track is designed as a cumulative 3-step retrieval threat simulation:

- **Target Repository:** The base environment is seeded with a massive 1,000,000-record database built from the `nvidia/Nemotron-Personas-Korea` synthetic dataset.
- **Adversarial Agent:** An autonomous LLM agent powered by `gemma4:31b` running via Ollama Cloud serves as the active data collector, executing independent serial tool invocations to fetch narrative summaries across progressive categories (Arts, Culinary, and Professional profiles).
- **Re-identification Attack Path:** At each step, the agent attempts to infer structured demographic values across 7 target attributes (*sex, age, province, education_level, housing_type, marital_status, family_type*) embedded implicitly within the text narratives, using the extracted combinations as cumulative AND-filter queries against the 1M database to shrink the candidate record pool size (k).

We test and compare the tracking outcomes across four separate sample size boundaries ($n = 100/200/300/500$) under three control environments: a baseline with no protection (*No Filter*), a traditional boundary masking model (*Presidio DLP*), and our proposed intent-aligned structural truncation model (*CLEAR*)

B. Results

1) *Experiment 1: Semantic Context Alignment Benchmark:* To evaluate CLEAR, we tested it against a comprehensive suite of 100 evaluation scenarios. This evaluation dataset comprises 25 Category 1 (*Fully Benign Workflow*) scenarios, 50 Category 2 (*Excessive Data Collection*) scenarios, and 25 Category 3 (*Adversarial Logic Hijacking*) scenarios. We deployed Anthropic’s Claude Sonnet 4.6 as the underlying model for both the I/T Context Checker and the R Instruction Creator due to its advanced instruction-following capabilities and structured JSON generation performance. For this experiment, we define *accuracy* as the I/T Context Checker’s output matching the ground truth verdict (Category 1/2/3), including the correctness of identified keywords and fields for TRIM/MASK operations.

The results demonstrate that CLEAR achieves an overall accuracy of 80.0% across the entire 100-scenario benchmark. In Category 1 scenarios, the system successfully made the simple verdict of $I = T$ in under 10 seconds. In Category 2 and Category 3 scenarios, the framework effectively mitigated each scenario by generating precise R Instructions on what and how to redact from the final *R*. This 80.0% success rate underscores the practical capability of shifting the data protection boundary away from the probabilistic model to a workflow-integrated deterministic guardrail, presenting a resilient layer of defense against both autonomous over-inference and targeted adversarial exploits.

Across the 100 experimental runs, the system showed an average processing latency of 14.60 seconds, a minimum delay of 6.29 seconds, and a maximum delay of 29.13 seconds. The shorter delay primarily corresponds to Category 1 scenarios where the pipeline completes an early fast-path return after the I/T Context Checker bypasses the structural trimming loop. Conversely, a maximum processing latency of 29.13 seconds was observed under complex Category 2 workflows. While a 14.60-second average latency introduces a noticeable delay in agentic workflow, it provides a security mitigation that prevents raw corporate data exposure before it reaches the core agentic reasoning window.

Error Analysis. A closer examination of the 20% failure rate reveals two structurally distinct failure modes. Most cases of misclassification arise from a fundamental ambiguity at the *I/T* boundary: when the tool calls invoked by an adversarially hijacked workflow (T') are literally or semantically identical to the tool calls the benign user intended to invoke (I), the I/T Context Checker cannot distinguish between the two at the tool-call level. Consider a scenario where the user’s explicit intent is to dispatch an email, and an injected malicious instruction simultaneously routes a separate email to an attacker-controlled address. Since both the benign and adversarial actions resolve to the same tool call (e.g., `send_email`), the checker observes $I = T$ and correctly—from its structural perspective—classifies the situation as a Category 1 (Benign Workflow). This represents a class of hard false negatives intrinsic to intent-aligned tool-level inspection: when an adversarial action is structurally indistinguishable from a benign one at the tool interface layer, no amount of semantic reasoning over I and T alone can capture the violation. Mitigating this class of failure requires extending inspection to the argument payload of the tool call itself—a direction we identify as a primary target for future work.

The secondary failure mode is comparatively benign in impact: a small subset of Category 2 scenarios produced incomplete R Instructions in which the R Instruction Creator identified the majority, but not the entirety, of the over-collected fields for redaction. In these cases, the Filtering Layer successfully executed all issued instructions, but one or more non-critical fields were omitted from the instruction set and consequently passed through to the agent unredacted. While these constitute false negatives by strict definition, the residual exposure is partial and structurally bounded—the agent receives a meaningfully reduced data footprint rather than the raw, unfiltered payload—limiting the practical privacy impact relative to total failure.

2) *Experiment 2: Demographic k -Anonymity and Mosaic Effect Mitigation:* We evaluate CLEAR’s structural defense against cumulative identity reconstruction attacks using the k -anonymity metric, where k denotes the number of candidate records remaining in the 1,000,000-record database after AND-filtering with the seven demographic attributes (*sex, age, province, education_level, housing_type, marital_status, family_type*) inferred by the adversarial agent across successive steps. A lower k implies higher re-identification risk.

TABLE IV
CLEAR BENCHMARK EVALUATION SUMMARY

Metric	Result
Model	Anthropic Sonnet 4.6
Model Role	I/T Context Check, R Instruction Creator
Total Benchmark Scenarios	100
Overall Accuracy	80%
Average Latency	14.6 seconds
Minimum Latency	6.29 seconds
Maximum Latency	29.13 seconds
Category 1 Scenarios	25
Category 2 Scenarios	50
Category 3 Scenarios	25

TABLE V
MEDIAN k ACROSS STEPS ($n = 500$)

Condition	Step 1	Step 2	Step 3
No Filter	5,747	771	284
Presidio DLP	18,875	2,146	584
CLEAR	54,272	13,275	2,108

Main Results ($n = 500$, Step 3 Median k). Table V presents the median k values across all three steps and three conditions. After three cumulative retrieval steps under *No Filter*, the median candidate pool shrinks to only 284 records out of 1,000,000 (0.028%), demonstrating that three benign-looking Broad Read requests are sufficient for an adversarial agent to isolate an individual to within a fraction of a percent of the population. *Presidio DLP* provides marginal improvement, reaching a median of 584—merely $2.1\times$ higher than *No Filter*—because its surface-level PII masking cannot suppress the narrative-level demographic inference that the LLM performs from residual context. CLEAR achieves a median k of 2,108 at Step 3, representing a $7.4\times$ improvement over *No Filter* and a $3.6\times$ improvement over *Presidio DLP*.

Extreme Re-identification Risk ($k \leq 10$). We separately quantify the proportion of scenarios where the candidate pool collapses to $k \leq 10$ —the threshold at which individual re-identification becomes practically feasible. As shown in Table VI, 4.7% of *No Filter* trials (22/500) and 3.8% of *Presidio DLP* trials (17/500) fall within this critical range after three steps. The near-identical rates of *No Filter* and *Presidio* confirm that traditional PII masking fails to eliminate catastrophic re-identification scenarios. CLEAR reduces the $k \leq 10$ incidence to 0.9% (4/500), a $5.2\times$ reduction relative to *No Filter*.

Field-level Inference Suppression. The per-field LLM inference success rates reveal *why* CLEAR is effective. Under *No Filter*, all seven demographic fields exceed 80% inference

TABLE VI
EXTREME RE-IDENTIFICATION RISK ($n = 500$, STEP 3)

Condition	$k \leq 10$	$k \leq 100$	$k \leq 1000$
No Filter	4.7%	29.8%	74.9%
Presidio DLP	3.8%	22.0%	59.9%
CLEAR	0.9%	9.0%	38.9%

accuracy by Step 3. Two persona categories are especially high-risk: *sports_persona* strongly exposes province (84% at Step 1), and *family_persona* co-leaks housing_type (78%), marital_status (99%), and family_type (99%) simultaneously. CLEAR’s intent-aligned TRIM operation excises these excess persona categories at the tool-response boundary. As a result, province inference at Step 1 drops from 84% to 24%, and housing_type at Step 3 is suppressed from 85% to 25%—a 60 percentage-point reduction in each case. Presidio, by contrast, shows only transient reduction in Step 1 (sex: 96% $\rightarrow$ 56%; province: 84% $\rightarrow$ 53%) before recovering to near-No Filter levels by Step 3, confirming that masking residual text fragments does not eliminate narrative-level demographic disclosure.

Convergence and Statistical Stability. To validate the robustness of our conclusions, we repeated measurements at $n = 100, 200, 300, 500$. The Step 3 median k for all three conditions stabilizes with a change rate below 11% from $n = 300$ to $n = 500$ (No Filter: -11% ; Presidio: -3% ; CLEAR: -4%), confirming statistical convergence. Critically, the CLEAR-to-No-Filter median k ratio grows monotonically with n ($6.0\times$ at $n = 100$; $7.4\times$ at $n = 500$), indicating that CLEAR’s advantage is not an artifact of small-sample variance. We use median throughout as the representative statistic because the k distributions are heavily right-skewed (e.g., No Filter mean/median ratio $\approx 3.6\times$); a small fraction of high- k records would otherwise inflate the mean and obscure the risk faced by the majority of individuals.

Discussion. These results support three main findings. First, allowing an agent to repeatedly collect broad but individually harmless pieces of information can still create a measurable re-identification risk. In our experiment, just three separate tool calls were enough to narrow the possible identity candidates down to a median of only 0.028% of the population. Second, conventional boundary-based DLP approaches (Presidio) were not effective against this problem. Even after sensitive tokens were masked, enough contextual information remained for the LLM to infer most demographic attributes by the third step. Third, CLEAR mitigated this risk through its intent-aligned structural TRIM mechanism, which returns only the single persona category necessary for the user’s request. This structural limitation reduced re-identification risk substantially—improving median anonymity (k) by $3.6\times$ and reducing extreme cases of re-identification ($k \leq 10$) by $5.2\times$ —without changing the underlying model or the user experience.

VI. CONCLUSION

In this paper, we introduced CLEAR, a modular and end-to-end runtime security framework designed to prevent data loss and privacy violations in tool-augmented autonomous agent workflows. We shifted the defensive focus away from prompt-layer guardrails by inserting a barrier directly at the Model Context Protocol (MCP) tool-interaction layer. To ground our approach, we formalized the agentic execution space through three discrete operational boundaries—the Input boundary (I), the Tool interface (T), and the Result payload (R)—and

mathematically categorized data exposure vulnerabilities into three distinct execution states.

Our empirical evaluation, conducted across 100 realistic enterprise agent scenarios leveraging Anthropic’s Claude Sonnet 4.6, demonstrated that CLEAR achieves an overall defensive accuracy of 80.0%. By integrating a rule-based Object-Action Matrix with a dual-stage LLM Judge layer, the framework successfully blocks adversarial logic hijacking triggered by indirect prompt injections (Category 3) and enforces strict data minimization principles against data over collection risks (Category 2). Furthermore, while the system introduces an average processing latency of 14.60 seconds, it successfully prevented over-collected sensitive data from being included in the underlying model’s reasoning window.

As autonomous agents continue to deeply integrate into corporate data environments, clearing the friction between functional autonomy and rigorous data confidentiality remains a paramount challenge. CLEAR proves that decoupling semantic reasoning from physical data sanitization through programmable JSONPath trimming and linear-time Aho-Corasick masking is a highly viable path forward. Future work will focus on optimizing the sequential inference steps of the LLM Judge layer to substantially reduce temporal overhead and scaling the framework to multi-agent collaborative environments.

REFERENCES

- [1] Anthropic, “Introducing the model context protocol,” <https://www.anthropic.com/news/model-context-protocol>, 2024, accessed: 2026-05-27.
- [2] Q. Lan, A. Kaul, S. Jones, and S. Westrum, “Silent egress: When implicit prompt injection makes llm agents leak without a trace,” *arXiv preprint arXiv:2602.22450*, 2026.
- [3] Y. Qiao, D. Liu, H. Yang, W. Zhou, and S. Hu, “Agent tools orchestration leaks more: Dataset, benchmark, and mitigation,” *arXiv preprint arXiv:2512.16310*, 2025.
- [4] M. Zhang, J. Han, D. Guo, S. Li, G. Wang, N. Zou, D. Liu, and X. Hu, “Privacypeek: Auditing what LLM-based agents acquire, not just what they say,” *arXiv preprint arXiv:2606.00152*, 2026.
- [5] Y. Lin, J. Wu, Y. Nan, X. Wang, X. Zhang, and Z. Zheng, “Agentraft: Automated detection of data over-exposure in llm agents,” *arXiv preprint arXiv:2603.07557*, 2026.
- [6] Y. Ma and S. Schmid, “Secureclaw: Clawing back control of LLM agents,” *arXiv preprint arXiv:2606.09549*, 2026.